

# **Competitive Programming**

**Section 33 Editorial: Inclusion-Exclusion**

Competitive Programming Course

2026-08-12

# Table of contents

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>Editorial overview</b>                       | <b>4</b>  |
| <b>2</b> | <b>A. Multiples Union</b>                       | <b>5</b>  |
| 2.1      | Restatement and diagnosis . . . . .             | 5         |
| 2.2      | Derivation . . . . .                            | 5         |
| 2.3      | Complete trace . . . . .                        | 5         |
| 2.4      | Algorithm and overflow invariant . . . . .      | 5         |
| 2.5      | Correctness proof . . . . .                     | 6         |
| 2.6      | Complexity, language notes, and tests . . . . . | 6         |
| 2.7      | Full C++ solution . . . . .                     | 6         |
| 2.8      | Full Python solution . . . . .                  | 7         |
| <b>3</b> | <b>B. All Symbols</b>                           | <b>9</b>  |
| 3.1      | Restatement and diagnosis . . . . .             | 9         |
| 3.2      | Derivation and trace . . . . .                  | 9         |
| 3.3      | Algorithm . . . . .                             | 9         |
| 3.4      | Correctness proof . . . . .                     | 10        |
| 3.5      | Complexity, language notes, and tests . . . . . | 10        |
| 3.6      | Full C++ solution . . . . .                     | 10        |
| 3.7      | Full Python solution . . . . .                  | 11        |
| <b>4</b> | <b>C. Bounded Candies</b>                       | <b>13</b> |
| 4.1      | Restatement and diagnosis . . . . .             | 13        |
| 4.2      | Derivation and trace . . . . .                  | 13        |
| 4.3      | Algorithm . . . . .                             | 13        |
| 4.4      | Correctness proof . . . . .                     | 14        |
| 4.5      | Complexity, language notes, and tests . . . . . | 14        |
| 4.6      | Full C++ solution . . . . .                     | 14        |
| 4.7      | Full Python solution . . . . .                  | 15        |
| <b>5</b> | <b>Local workflow</b>                           | <b>17</b> |
| <b>6</b> | <b>D. Seating Without Matches</b>               | <b>18</b> |
| 6.1      | Restatement . . . . .                           | 18        |
| 6.2      | Constraints and traps . . . . .                 | 18        |
| 6.3      | Bad events and reduction . . . . .              | 18        |
| 6.4      | Worked intersection trace . . . . .             | 18        |
| 6.5      | Algorithm . . . . .                             | 19        |
| 6.6      | Correctness proof . . . . .                     | 19        |
| 6.7      | Complexity . . . . .                            | 19        |
| 6.8      | Full C++ solution . . . . .                     | 19        |
| 6.9      | C++ implementation notes . . . . .              | 20        |
| 6.10     | Full Python solution . . . . .                  | 20        |
| 6.11     | Python implementation notes . . . . .           | 21        |
| 6.12     | Traps and tests . . . . .                       | 21        |

|          |                              |           |
|----------|------------------------------|-----------|
| <b>7</b> | <b>Stress testing</b>        | <b>22</b> |
| <b>8</b> | <b>What to carry forward</b> | <b>23</b> |

# 1 Editorial overview

| Problem            | Bad or union events             | Intersection count     | Cost               |
|--------------------|---------------------------------|------------------------|--------------------|
| A. Multiples Union | divisible by a listed divisor   | $N / \text{LCM}$       | $O(m 2^m)$         |
| B. All Symbols     | a named symbol is missing       | $(k-j)^n$              | $O(q k \log n)$    |
| C. Bounded Candies | a child receives at least $b+1$ | shifted stars and bars | about $10^6$ terms |

For every problem, first count one selected intersection. The sign is only the last step.

## 2 A. Multiples Union

### 2.1 Restatement and diagnosis

Count values  $x$  in  $[1, N]$  divisible by at least one input divisor. Here  $m$  is at most 20, while  $N$  and every divisor can be  $10^{18}$ . Scanning  $x$  is impossible; enumerating  $2^m$  subsets is intended. Duplicated divisors describe identical events, so sort and deduplicate before the subset loop.

### 2.2 Derivation

For selected divisors  $S$ ,  $x$  belongs to every chosen event precisely when its value is a multiple of  $L = \text{lcm}(S)$ . The intersection has  $\text{floor}(N/L)$  values. Add this count for an odd number of selected divisors and subtract it for an even number.

### 2.3 Complete trace

For  $N=20$  and divisors 2, 3, and 5:

| set     | LCM | signed count | running answer |
|---------|-----|--------------|----------------|
| {2}     | 2   | +10          | 10             |
| {3}     | 3   | + 6          | 16             |
| {5}     | 5   | + 4          | 20             |
| {2,3}   | 6   | - 3          | 17             |
| {2,5}   | 10  | - 2          | 15             |
| {3,5}   | 15  | - 1          | 14             |
| {2,3,5} | 30  | + 0          | 14             |

The final answer is 14. Number 6 has coefficient +1 for {2}, +1 for {3}, and -1 for {2,3}, hence it is counted exactly once.

### 2.4 Algorithm and overflow invariant

The invariant while building one subset is:  $L$  is its exact LCM unless the subset has already been proved to contribute zero. Update an LCM with  $L/\text{gcd}(L,d)*d$ , but never perform a multiplication that exceeds  $N$ :

```

sort and erase duplicate divisors
answer = 0
for every nonempty subset:
    L = 1; bits = 0; nonzero = true
    for each selected divisor d:
        bits += 1
        g = gcd(L,d)
        if L/g > N/d:
            nonzero = false
            break
        L = L/g*d
    if nonzero:
        add N/L when bits is odd, otherwise subtract N/L

```

Once L exceeds N, every later LCM is a multiple of L and also exceeds N. The intersection is empty, so stopping is both safe and necessary to prevent overflow.

## 2.5 Correctness proof

Let  $E_i$  be the set of numbers in  $[1, N]$  divisible by divisor  $i$ . The wanted set is the union of the  $E_i$ . For every selected subset, the algorithm computes the size of its exact intersection using the subset LCM, then uses the inclusion-exclusion sign for its size. Therefore each number in the union has total coefficient one and each number outside has coefficient zero. The capped LCM branch skips only empty intersections. Thus the answer is correct. QED.

## 2.6 Complexity, language notes, and tests

Time is  $O(m 2^m)$ , memory  $O(m)$ . In C++, long long is enough with the guarded comparison; `__int128` is an equally clear alternative. Python integers do not overflow, but should still stop an empty subset early.

Wrong answers to target:

Adding  $N/d$  independently overcounts common multiples.  
 Multiplying an LCM before checking can overflow at  $10^{18}$ .  
 Leaving duplicates in the event list double counts the same condition.

Test divisor 1, a divisor greater than N, duplicate divisors,  $N=1$ , and an LCM just larger than N.

## 2.7 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    int m;
    if (!(cin >> n >> m)) return 0;
    vector<long long> d(m);
    for (long long &x : d) cin >> x;

    long long answer = 0;
    for (int mask = 1; mask < (1 << m); mask++) {
        __int128 lcm = 1;
        int bits = 0;
        bool ok = true;
        for (int i = 0; i < m; i++) {
            if (mask & (1 << i)) {
                bits++;
                long long g = gcd((long long)min<__int128>(lcm, LLONG_MAX), d[i]);
                lcm = lcm / g * d[i];
                if (lcm > n) {
                    ok = false;
                    break;
                }
            }
        }
        if (!ok) continue;
        long long count = n / (long long)lcm;
        if (bits % 2) answer += count;
        else answer -= count;
    }
    cout << answer << '\n';
    return 0;
}

```

## 2.8 Full Python solution

```

import math
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return

```

```

n, m = data[0], data[1]
d = data[2 : 2 + m]
answer = 0
for mask in range(1, 1 << m):
    lcm = 1
    bits = 0
    ok = True
    for i in range(m):
        if (mask >> i) & 1:
            bits += 1
            g = math.gcd(lcm, d[i])
            lcm = lcm // g * d[i]
            if lcm > n:
                ok = False
                break
    if not ok:
        continue
    count = n // lcm
    answer += count if bits % 2 == 1 else -count
print(answer)

if __name__ == "__main__":
    main()

```

## 3 B. All Symbols

### 3.1 Restatement and diagnosis

For each query, count length- $n$  strings over  $k$  symbols in which every symbol appears. The bounds  $n \leq 10^9$ ,  $k \leq 500$ ,  $q \leq 200$  say to loop over  $k$  and use binary exponentiation in  $n$ .

### 3.2 Derivation and trace

Let  $E_i$  mean that symbol  $i$  is missing. If  $j$  particular symbols are missing, there are  $(k-j)^n$  strings over the remaining symbols, and  $C(k,j)$  ways to choose those missing symbols. Starting from all strings gives

$$\text{answer} = \sum_{j=0}^k (-1)^j C(k,j) (k-j)^n.$$

For  $n=3$ ,  $k=3$ :

```
j=0: + C(3,0) 3^3 = +27
j=1: - C(3,1) 2^3 = -24
j=2: + C(3,2) 1^3 = + 3
j=3: - C(3,3) 0^3 = - 0
answer = 6
```

A string `aaa` is counted once, removed three times, restored three times, and removed once. Its coefficient is zero. A string `abc` is in no missing-symbol event and remains once.

### 3.3 Algorithm

Precompute factorials and inverse factorials through the maximum  $k$ , not through  $n$ . All indices are at most 500 and therefore invertible modulo the prime MOD.

```
read all queries and prepare factorial tables through max_k
for each n,k:
    ans = 0
    for missing from 0 to k:
        term = C(k,missing) * pow_mod(k-missing,n)
        add if missing is even; subtract otherwise
    normalize ans modulo MOD
```

Use the conventional binary-exponentiation value  $0^0=1$ . It makes the local case  $n=0,k=1$  evaluate to zero: the empty string contains no symbol.

### 3.4 Correctness proof

Each subset of  $j$  missing symbols has exactly  $(k-j)^n$  strings, and there are  $C(k,j)$  subsets of that size. Inclusion-exclusion subtracts the union of all missing-symbol events from the full set of strings. The survivors are exactly the strings in which every symbol occurs. Valid factorial inverses compute the binomial coefficients modulo MOD. QED.

### 3.5 Complexity, language notes, and tests

Preprocessing costs  $O(\max k)$ ; each query costs  $O(k \log n)$  and memory is  $O(\max k)$ . C++ products need long long. Python must use `pow(base, exponent, MOD)`, not an enormous ordinary power.

Test  $n < k$ ,  $n = k$ ,  $k = 1$ ,  $n = 0$ , and a huge  $n$  with small  $k$ .

Common failures are using  $C(n,j)$ , computing powers linearly, stopping the loop at  $n$  rather than  $k$ , and failing to normalize a subtraction.

### 3.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    vector<pair<long long, int>> queries(q);
    int limit = 0;
    for (auto &[n, k] : queries) {
        cin >> n >> k;
        limit = max(limit, k);
    }
    vector<long long> fact(limit + 1, 1), invfact(limit + 1, 1);
```

```

for (int i = 1; i <= limit; i++) fact[i] = fact[i - 1] * i % MOD;
invfact[limit] = mod_pow(fact[limit], MOD - 2);
for (int i = limit; i >= 1; i--) invfact[i - 1] = invfact[i] * i % MOD;
auto C = [&](int n, int r) -> long long {
    if (r < 0 || r > n) return 0;
    return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD;
};
for (auto [n, k] : queries) {
    long long ans = 0;
    for (int missing = 0; missing <= k; missing++) {
        long long term = C(k, missing) * mod_pow(k - missing, n) % MOD;
        if (missing % 2) ans = (ans - term + MOD) % MOD;
        else ans = (ans + term) % MOD;
    }
    cout << ans << '\n';
}
return 0;
}

```

### 3.7 Full Python solution

```

import sys

MOD = 1_000_000_007

def prepare(limit: int) -> tuple[list[int], list[int]]:
    fact = [1] * (limit + 1)
    for i in range(1, limit + 1):
        fact[i] = fact[i - 1] * i % MOD
    invfact = [1] * (limit + 1)
    invfact[limit] = pow(fact[limit], MOD - 2, MOD)
    for i in range(limit, 0, -1):
        invfact[i - 1] = invfact[i] * i % MOD
    return fact, invfact

def comb(n: int, r: int, fact: list[int], invfact: list[int]) -> int:
    if r < 0 or r > n:
        return 0
    return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return

```

```

q = data[0]
queries = [(data[i], data[i + 1]) for i in range(1, 2 * q + 1, 2)]
fact, invfact = prepare(max(k for _, k in queries))
out = []
for n, k in queries:
    ans = 0
    for missing in range(k + 1):
        term = comb(k, missing, fact, invfact) * pow(k - missing, n, MOD) % MOD
        if missing % 2:
            ans -= term
        else:
            ans += term
    out.append(str(ans % MOD))
print("\n".join(out))

if __name__ == "__main__":
    main()

```

## 4 C. Bounded Candies

### 4.1 Restatement and diagnosis

Count nonnegative solutions  $x_1 + \dots + x_n = s$  with every  $x_i$  at most  $b$ . Here  $n$ ,  $s$ , and  $b$  are at most 5000. Across all queries, the sum of  $\min(n, \text{floor}(s/(b+1)))$  is at most one million, which is the cost budget for the nonzero terms.

### 4.2 Derivation and trace

Without upper bounds there are  $C(s+n-1, n-1)$  solutions. For a selected set of  $j$  children that receive at least  $b+1$ , remove  $b+1$  candies from every selected child. This is a bijection to unrestricted distributions of  $s-j(b+1)$  candies, so the intersection count is

$$C(s-j(b+1)+n-1, n-1).$$

For  $n=3$ ,  $s=4$ ,  $b=2$ :

```
j=0: + C(3,0) C(6,2) = +15
j=1: - C(3,1) C(3,2) = - 9
j=2: remaining sum is negative, stop
answer = 6
```

The invalid allocation  $(3,1,0)$  appears in the unrestricted count and once in the event for child one, so it cancels. The valid allocation  $(2,1,1)$  appears only in the unrestricted count.

### 4.3 Algorithm

Precompute factorials through the largest  $n+s-1$  over all queries. The local index is at most 9999, far below MOD.

```
for each n,s,b:
    step = b+1
    last = min(n, floor(s/step))
    ans = 0
    for bad from 0 to last:
        remaining = s-bad*step
        term = C(n,bad) * C(remaining+n-1,n-1)
        add term for even bad, subtract for odd bad
    print normalized ans
```

The loop has  $\text{last}+1$  iterations. Since  $b$  is nonnegative,  $\text{step}$  is at least one. The aggregate contract makes this efficient in Python as well as C++.

## 4.4 Correctness proof

Take  $E_i$  to mean  $x_i$  is at least  $b+1$ . For every selected set of bad children, the subtraction transformation is bijective, hence the algorithm has the exact intersection size.  $C(n, \text{bad})$  groups all selected sets of the same size. Inclusion-exclusion makes an invalid allocation's total coefficient zero and a valid allocation's coefficient one. The result is exactly the number of bounded allocations. QED.

## 4.5 Complexity, language notes, and tests

Preprocessing is  $O(\max(n+s))$ ; one query is  $O(\min(n, \text{floor}(s/(b+1)))+1)$ , with  $O(\max(n+s))$  memory. In C++, use long long products and normalize every subtraction. Python may normalize once at the end.

Test  $s=0$ ,  $b=0$ ,  $n=1$ ,  $b \geq s$ , and  $n=2, s=5, b=2$ . Typical bugs shift by  $b$  instead of  $b+1$ , continue after the remaining sum is negative, or precompute only through  $n$  instead of  $n+s-1$ .

## 4.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    vector<array<int, 3>> queries(q);
    int limit = 0;
    for (auto &cur : queries) {
        cin >> cur[0] >> cur[1] >> cur[2];
        limit = max(limit, cur[0] + cur[1]);
    }
    vector<long long> fact(limit + 1, 1), invfact(limit + 1, 1);
    for (int i = 1; i <= limit; i++) fact[i] = fact[i - 1] * i % MOD;
```

```

invfact[limit] = mod_pow(fact[limit], MOD - 2);
for (int i = limit; i >= 1; i--) invfact[i - 1] = invfact[i] * i % MOD;
auto C = [&](int n, int r) -> long long {
    if (r < 0 || r > n) return 0;
    return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD;
};
for (auto [n, s, b] : queries) {
    long long ans = 0;
    int step = b + 1;
    for (int bad = 0; bad <= n && bad * step <= s; bad++) {
        int remaining = s - bad * step;
        long long term = C(n, bad) * C(remaining + n - 1, n - 1) % MOD;
        if (bad % 2) ans = (ans - term + MOD) % MOD;
        else ans = (ans + term) % MOD;
    }
    cout << ans << '\n';
}
return 0;
}

```

## 4.7 Full Python solution

```

import sys

MOD = 1_000_000_007

def prepare(limit: int) -> tuple[list[int], list[int]]:
    fact = [1] * (limit + 1)
    for i in range(1, limit + 1):
        fact[i] = fact[i - 1] * i % MOD
    invfact = [1] * (limit + 1)
    invfact[limit] = pow(fact[limit], MOD - 2, MOD)
    for i in range(limit, 0, -1):
        invfact[i - 1] = invfact[i] * i % MOD
    return fact, invfact

def comb(n: int, r: int, fact: list[int], invfact: list[int]) -> int:
    if r < 0 or r > n:
        return 0
    return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:

```

```

        return
q = data[0]
queries = [(data[i], data[i + 1], data[i + 2]) for i in range(1, 3 * q + 1, 3)]
limit = max(n + s for n, s, _ in queries)
fact, invfact = prepare(limit)
out = []
for n, s, b in queries:
    ans = 0
    step = b + 1
    for bad in range(0, min(n, s // step) + 1):
        remaining = s - bad * step
        term = comb(n, bad, fact, invfact) * comb(remaining + n - 1, n - 1, fact, invfact)
        ans += term if bad % 2 == 0 else -term
    out.append(str(ans % MOD))
print("\n".join(out))

if __name__ == "__main__":
    main()

```

## 5 Local workflow

Run:

```
CP_TARGET=solution python3 sections/33_inclusion_exclusion/check.py
```

For small random cases, compare against direct scanning for A, short-string enumeration for B, and recursive allocation enumeration for C. This catches a wrong inclusion-exclusion sign much more reliably than samples do.

## 6 D. Seating Without Matches

### 6.1 Restatement

Count the ways to seat  $n$  labeled people in  $n$  labeled seats so that nobody sits in their own seat. This is the derangement count, requested modulo the prime 1\_000\_000\_007.

### 6.2 Constraints and traps

$n$  can reach 1\_000\_000, so enumerating  $n!$  seatings is impossible. The factorial arrays must therefore be linear, and all division in the formula is modular division.

```
n = 1 has answer 0: its only seating fixes that person.  
The inclusion-exclusion sign is positive for k = 0.  
Subtracting a term in C++ needs normalization back into [0, MOD).  
The factorial bound is below MOD, so every fact[k] for k <= n is invertible.
```

### 6.3 Bad events and reduction

Let event  $E_i$  mean person  $i$  remains in seat  $i$ . We want permutations in none of these events. For any chosen set of  $k$  fixed people, the remaining  $n-k$  people can be seated in  $(n-k)!$  ways. There are  $C(n,k)$  chosen sets, so inclusion-exclusion gives:

```
answer = sum(k=0..n) (-1)^k C(n,k) (n-k)!  
        = sum(k=0..n) (-1)^k n! / k!
```

All arithmetic is modulo prime MOD, so compute factorials and inverse factorials with Fermat inverses.

For  $n=4$ , the alternating terms are  $24 - 24 + 12 - 4 + 1 = 9$ .

### 6.4 Worked intersection trace

For three people, define  $E_1, E_2, E_3$ . There are  $3! = 6$  unrestricted seatings. Each individual event, such as  $E_1$ , fixes one person and leaves two people free, hence has  $2! = 2$  seatings. The three single-event counts subtract  $3 * 2 = 6$ .

Any two fixed-seat events force the third person as well, so each pair intersection has  $1! = 1$  seating and the three pair intersections add 3. The triple intersection also has one seating and is subtracted:

$$6 - 3*2 + 3*1 - 1 = 2$$

The two remaining seatings are the two 3-cycles. More generally, choosing a specific set of  $k$  events forces exactly those  $k$  people, leaving  $(n-k)!$  arrangements; there are  $C(n,k)$  ways to choose the set.

## 6.5 Algorithm

```
precompute fact[0..n] and inv_fact[0..n]
answer = 0
for k from 0 to n:
    term = fact[n] * inv_fact[k]
    add term for even k; subtract it for odd k
print normalized answer
```

## 6.6 Correctness proof

**Lemma.** The intersection of any selected  $k$  fixed-seat events contains  $(n-k)!$  permutations.

**Proof.** Those  $k$  people and seats are forced. The remaining people may be permuted freely among remaining seats.  $\square$

**Theorem.** The algorithm returns the number of seatings with no original seat match.

**Proof.** Inclusion-exclusion over events  $E_i$  adds and subtracts the intersection counts from the lemma. A permutation with one or more fixed seats has total coefficient zero; a permutation with none has coefficient one. The factorial formula computes each grouped term modulo MOD.  $\square$

## 6.7 Complexity

Time:  $O(n)$   
Memory:  $O(n)$

## 6.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;
long long power(long long a, long long b) {
    long long r = 1;
    for (; b; b >>= 1, a = a * a % MOD) if (b & 1) r = r * a % MOD;
    return r;
}
```

```

}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n; cin >> n;
    vector<long long> fact(n + 1), inv_fact(n + 1);
    fact[0] = 1;
    for (int i = 1; i <= n; ++i) fact[i] = fact[i - 1] * i % MOD;
    inv_fact[n] = power(fact[n], MOD - 2);
    for (int i = n; i; --i) inv_fact[i - 1] = inv_fact[i] * i % MOD;
    long long answer = 0;
    for (int k = 0; k <= n; ++k) {
        long long term = fact[n] * inv_fact[k] % MOD;
        answer = (answer + (k & 1 ? -term : term) + MOD) % MOD;
    }
    cout << answer << "\\n";
}

```

## 6.9 C++ implementation notes

Compute  $\text{inv\_fact}[n] = \text{fact}[n]^{-(\text{MOD}-2)}$  once, then fill inverse factorials downward with  $\text{inv\_fact}[i-1] = \text{inv\_fact}[i] * i$ . Normalize each alternating update immediately with  $+$  MOD before  $\% \text{MOD}$ , rather than allowing a negative C++ remainder to escape.

## 6.10 Full Python solution

```

import sys
MOD = 1_000_000_007

def main() -> None:
    n = int(sys.stdin.buffer.read())
    fact = [1] * (n + 1)
    for i in range(1, n + 1):
        fact[i] = fact[i - 1] * i % MOD
    inv_fact = [1] * (n + 1)
    inv_fact[n] = pow(fact[n], MOD - 2, MOD)
    for i in range(n, 0, -1):
        inv_fact[i - 1] = inv_fact[i] * i % MOD
    print(sum((( -1 if k & 1 else 1) * fact[n] * inv_fact[k]) for k in range(n + 1)) % MOD)

if __name__ == "__main__":
    main()

```

## 6.11 Python implementation notes

Python's final `% MOD` normalizes a negative alternating sum automatically. The list comprehensions are not needed here: fill the two factorial arrays in loops so their  $O(n)$  storage and indexing remain explicit.

## 6.12 Traps and tests

- The sign starts positive at  $k=0$ .
- $n! / k!$  is modular multiplication by an inverse factorial, not integer division after reduction.
- Test  $n=1$ ,  $n=2$ , and a large  $n$  for factorial preprocessing.

## 7 Stress testing

For  $n \leq 8$ , enumerate all permutations and directly reject those with a fixed position. This is independent of inclusion-exclusion. The scale oracle uses the separate derangement recurrence  $D[n] = (n-1)(D[n-1] + D[n-2])$  for  $n=1000000$ .

## 8 What to carry forward

For a “none of these conditions” count, define one bad event per condition, count a selected intersection, and group equal-size subsets only after that intersection is understood.